

Avance individual — Anthony (Tablet Flutter + Backend gaze tracking)

Five Nights at Freddy's: Attention Defense — CS2H1, Interacción Humano-Computador, UCSP
Insumo para el informe/presentación grupal del primer avance (no es el documento final de entrega)

1. QUÉ SE HIZO

1.1 Tablet — Flutter/Dart

Aplicación Flutter (paquete `fn_attention_defense_tablet`) que corre en la tablet del jugador.
Versión actual: `0.6.0`.

- Flujo completo Splash → Juego → Fin de partida / Victoria, funcionando de punta a punta.
- **10 mini-juegos/tareas implementados y jugables:** los 4 originales (conectar cables, girar perillas, resolver secuencias, ritmo crítico) más 6 nuevos agregados en la última sesión, tomados de una lista de tareas que se redactó para acercar la mecánica a un contexto más narrativo de vigilancia nocturna: reiniciar WiFi, encender ventiladores, reparar temperatura, procesar datos, subir datos, y trazar curso.
- Menú de tareas: el jugador elige en qué tarea entrar (en vez de un flujo lineal forzado tarea por tarea).
- **Rediseño de la mecánica de riesgo** (reemplaza la barra de vida que se desgastaba con daño fijo): ahora hay un *riesgo global 0-100%* que sube con el tiempo/checkpoints horarios y con tareas sin resolver, y baja al completarlas. Un ataque recibido es siempre fatal (game over inmediato), en vez de restar puntos de vida — se acerca más a cómo funciona el AI level real de Five Nights at Freddy's 2.
- Sistema de 5 noches con reloj simulado (12:00am–6:00am).
- Comunicación por WebSocket (paquete `web_socket_channel`) contra un servidor simulado (`MockServerService`), porque el backend real todavía no existía cuando se hizo esta parte.
- Manejo de estado con Provider; código con nombres de variables/clases en castellano (convención del equipo).

Nota: los 6 mini-juegos nuevos y el sistema de riesgo están implementados y conectados en la app, pero todavía no se probaron jugando (falta verificación interactiva).

1.2 Backend — Python (gaze tracking)

Prototipo de detección de atención visual: define si el jugador mira la pantalla (PC), la tablet, o ninguna de las dos, usando la cámara.

- **MediaPipe Face Mesh** (API `solutions.face_mesh`) + **OpenCV** para capturar cámara y detectar 468 puntos del rostro, incluido el iris.
- Señal combinada: posición del iris dentro del ojo (izquierda/derecha, arriba/abajo) + orientación de la cabeza (inclinación y giro), calculada con proporciones geométricas entre puntos fijos del rostro (frente, nariz, mentón, pómulos).
- Calibración por jugador: mira 5 puntos de su pantalla real (centro + 4 esquinas) y un punto de referencia "tablet" — define zonas válidas para cada estado en vez de usar un umbral fijo genérico.
- Filtro temporal (~15 cuadros seguidos) para que el estado no cambie de golpe por un parpadeo o un movimiento brusco de un solo frame.
- Código dividido en módulos (`landmarks`, `estimacion`, `calibracion`, `deteccion`) en vez de un solo script, para poder depurarlo por partes.

1.3 Backend — lógica del juego en Python (recién iniciado)

En paralelo se empezó el diseño orientado a objetos de la lógica de juego (fuera del gaze tracking): clases `Juego` y `Animatronico`. Todavía en etapa muy temprana, sin probar.

- `Animatronico`: nombre, imagen, sonido, habilidades, nivel de IA (escala 0-20, mismo concepto que el AI level de FNAF2 usado también del lado Flutter) y posición dentro de un mapa.
- El mapa se modela como un grafo (diccionario de nodos y sus adyacencias: escenario, áreas, nodos intermedios, jugador) — cada animatrónico se mueve por ese grafo con un roll de probabilidad según su nivel de IA.
- `Juego`: orquesta la partida (noche actual, duración, cantidad de animatrónicos) y corre un

loop que hace moverse a cada animatrónico cada 3-5 segundos.

2. TECNOLOGÍAS USADAS

Componente	Tecnología	Para qué
Tablet	Flutter 3.13+ / Dart 3.0+	UI de mini-juegos, multiplataforma Android/iOS
Tablet	web_socket_channel	Comunicación en tiempo real con el servidor
Tablet	Provider	Manejo de estado de la app
Backend	MediaPipe Face Mesh (Python)	Detección de 468 puntos faciales + iris en tiempo real
Backend	OpenCV (opencv-python)	Captura de cámara, dibujo de overlay, ventana de prueba
Backend	NumPy	Cálculo de proporciones geométricas
Backend	Python (POO puro, sin librerías)	Lógica de juego: clases <code>Juego</code> / <code>Animatronico</code> , movimiento por grafo

3. PROBLEMAS ENCONTRADOS

3.1 Backend / MediaPipe

Versión de MediaPipe rota: la última versión (1.0+) eliminó la API usada en los tutoriales/ejemplos (`mp.solutions.face_mesh`), reemplazándola por una API nueva (Tasks API) que exige descargar un modelo aparte. Solución: se fijó la versión `mediapipe==0.10.21`, la última que conserva la API anterior — evita reescribir todo el pipeline sin perder funcionalidad.

Cálculo de orientación de cabeza poco confiable al inicio: el primer método usado (`solvePnP` de OpenCV, con 6-10 puntos de referencia) daba ángulos que saltaban de golpe (de +150° a -150°) sin que la cabeza se hubiera movido tanto — es un problema conocido de ambigüedad cuando hay pocos puntos de referencia. Se reemplazó por un método más simple basado en proporciones de distancias entre puntos fijos del rostro (frente-nariz-mentón para inclinación, pómulos para giro), que resultó mucho más estable.

Confusión entre "mirar pantalla" y "mirar tablet": el problema más persistente. Al bajar la cabeza para mirar la tablet, los ojos tienden a compensar rotando hacia arriba (para seguir viendo la pantalla), lo que hacía que el sistema detectara "pantalla" aunque la cabeza estuviera agachada. Se resolvió combinando la señal de iris vertical con la de inclinación de cabeza en un solo "vector de mirada", en vez de tratarlas como reglas separadas.

Calibración con umbral fijo no funcionaba entre sesiones: un valor de umbral fijo (ej. "15 grados") no se sostenía entre una sesión y otra, porque la postura, distancia a la cámara y ángulo de la webcam cambian cada vez. Se resolvió agregando un paso de calibración por jugador (mirar puntos conocidos de la pantalla) en vez de usar un número fijo en el código.

3.2 Tablet / Flutter

Falta de servidor real: como el backend no estaba listo, todo el desarrollo de Flutter se hizo contra un servidor simulado (`MockServerService`) que imita las respuestas esperadas. Esto permitió avanzar en paralelo, pero implica que la integración real (protocolo JSON, tiempos de respuesta, reconexión) todavía no se probó de punta a punta.

Bug de sincronización al iniciar: la primera tarea simulada se enviaba antes de que la pantalla de juego alcanzara a suscribirse al canal de eventos, por lo que se perdía y la app quedaba esperando indefinidamente. Se corrigió retrasando el primer envío.

Extensiones de protocolo aún no acordadas con el equipo: el sistema de noches, el menú de tareas y el nuevo sistema de riesgo (`risk_percent`) requieren mensajes nuevos en el protocolo JSON que todavía no están validados con el backend/Unity — por ahora solo existen simulados del lado tablet. Si el compañero de backend ya venía trabajando sobre el protocolo original (con daño fijo por ataque), este cambio de mecánica puede generar incompatibilidad si no se coordina antes de integrar.

6 tareas nuevas + sistema de riesgo sin probar interactivamente: el código está escrito e integrado (compila, está conectado al flujo del juego), pero no se jugó todavía una partida completa con las tareas nuevas para confirmar que el balance del riesgo (qué tan rápido sube, cuánto baja por tarea completada) tenga sentido jugable.

3.3 Backend — lógica de juego (POO), recién empezado

Al ser código muy reciente y sin ejecutar todavía, aparecieron varios problemas típicos de una primera versión — se documentan porque son justamente el tipo de cosas que hay que resolver antes de integrar con el resto:

Import circular entre módulos: `juego.py` importa `Animatronico` desde `animatronicos.py`, y `animatronicos.py` a su vez importa el módulo `juego` completo sin usarlo — es un import muerto que además genera una dependencia circular entre ambos archivos, un problema que se vuelve más difícil de rastrear a medida que el proyecto crece.

Número de argumentos no coincide al crear un animatrónico: `Animatronico.__init__` pide 7 parámetros (incluyendo `posicion`), pero donde se crea el primer animatrónico (Freddy) dentro de `Juego` solo se pasan 6 — falta el argumento de posición. Esto va a lanzar un error en cuanto se intente correr el juego.

Referencia a un atributo que no existe: el método `moverse` de `Animatronico` hace referencia a `self.grafoMapa`, pero ese atributo nunca se asignó en el constructor — el grafo del mapa vive en la clase `Juego`, no en cada animatrónico. Falta decidir si cada animatrónico guarda su propia referencia al mapa o si se le pasa como parámetro cada vez que se mueve.

Estos tres puntos son evidencia de que, aunque cada parte (Flutter, gaze tracking, lógica de juego) avanza y funciona por separado en pruebas aisladas, **la integración entre las tres todavía no se probó junta** — es razonable esperar más errores de este tipo (protocolos que no calzan, referencias cruzadas entre módulos, timing entre procesos) cuando se junten de verdad.

4. CONCLUSIONES Y PRÓXIMOS PASOS (DE MI PARTE)

- El gaze tracking con MediaPipe es viable en tiempo real con una webcam común — no hace falta hardware especial.
- La parte más difícil del gaze tracking no fue la detección facial en sí (eso lo resuelve MediaPipe), sino traducir esa información en una señal confiable de "a qué le está prestando atención el jugador" — ahí se fueron la mayoría de las iteraciones.
- Del lado Flutter, cada pieza nueva (mini-juego, sistema de riesgo) se integra rápido al esqueleto ya armado, pero eso también significa que el protocolo JSON compartido va quedando desactualizado más rápido de lo que se coordina con el equipo — hay que sincronizar antes de que se acumulen más cambios sin validar.
- La lógica de juego en Python (animatrónicos, grafo del mapa) recién arranca y ya tiene errores de ejecución esperables en código nuevo — sirve como recordatorio de que integrar las tres partes (Flutter + gaze tracking + lógica de juego + Unity) va a sacar a la luz bastantes más problemas de los que aparecen probando cada parte por separado.
- Próximo paso backend (gaze tracking): conectar el estado detectado (pantalla/tablet/nada) a Unity por socket, y probar con la cámara USB en vez de la webcam integrada.
- Próximo paso backend (lógica de juego): corregir los errores encontrados, decidir cómo el grafo del mapa se comparte entre animatrónicos, y probar el loop de movimiento.
- Próximo paso Flutter: probar interactivamente las 6 tareas nuevas y el sistema de riesgo, y conectar contra el servidor Python real (cuando esté listo) para validar que el protocolo JSON acordado funcione igual que con el mock.
- Pendiente coordinar con el equipo: validar las extensiones de protocolo (sistema de noches, menú de tareas, riesgo global) antes de que se vuelvan permanentes de un solo lado.